2026-03-09：V1 阶段第三轮综合审计报告

> 审计对象： `openagentic-sdk-erlang` 当前 working tree（不是仅审 `HEAD` ）
> 审计时间：2026-03-09（Asia/Shanghai）
> 审计范围：
>
> 1. 与源项目的对齐程度：语义、功能、测试套件
> 2. 当前实现本身的审计：正确性、架构一致性、性能/可扩展性、测试充分性

Executive Summary

当前这棵 Erlang 工作树已经把 `case -> monitoring -> inspection -> reconsideration` 主链基本跑通，本地 EUnit 也重新验证为 `230 tests, 0 failures` ，说明 **v1 的主功能闭环已经可用**。[4][17]

但如果把“整个 v1 阶段”按完整 PRD 去审，而不是只按最近几轮 Phase 收口去审，那么结论不能写成“零差异完成”。真正的现状更准确地说是：**Phase 1~3 主链基本收口，Phase 4 的落盘/聚合/性能设计只做到了“写侧先补出来”，读侧和工程硬化还没有真正闭合**。[1][2][3][4][7][8][9]

与源项目对齐方面，需要分成两层看：

- **直接 parity 基线** 仍然是 Kotlin sibling，因为本仓 `AGENTS.md` 、 `scripts/kotlin-parity-check.ps1` 、以及大量模块注释都把 Kotlin 当作直接对齐对象；在这个层面，**默认工具集和大部分 schema 仍然同源**，但 **7 个 toolprompt 已经发生语义漂移**， `safe tools` 默认放行集合也扩大了，现有 parity helper 本身还落后于当前 Erlang 模块拆分。[5][6][13][15][18]
- **源头 lineage 基线** 是 Python `openagentic-sdk` 。它今天本地仍能跑出 `369 tests, OK` ，说明源头仓的核心 SDK 测试纪律仍然更成熟；Erlang 当前在 workflow/case-governance 上已经远超 Kotlin，但在通用 SDK 维度的 hooks/MCP/plugin/CLI 深水区覆盖，仍不等价于 Python 源仓。[5][20]

这轮审计里最需要立刻进入修复队列的不是“再加功能”，而是几个工程风险点：

1. **高危 atom 泄漏风险**：JSON 读盘后把任意 binary key 转成 atom，存在被异常数据打爆 atom table 的风险。[11]
2. **调度器阻塞风险**：scheduler 每 tick 全量扫 case/task，并在扫描线程里同步执行 `run_task -> runtime query` ，会把调度、IO、模型调用耦死在一条串行链上。[9][10]
3. **Inbox 正确性风险**： `update_mail_state/2` 当前会把一个 case 下的所有 mail 一起改状态，而不是只改目标 `mail_id` ，现有测试没有覆盖多 mail 场景。[8][23]
4. **Phase 4 设计未闭合**：系统已经写出了 indexes / timeline / operation，但 Web 查询仍然现场扫目录拼状态，导致读放大、写放大同时存在。[2][7][8][24]

结论一句话：这不是“继续扩 v1”的时点，而是“把 v1 从可跑，推进到可长期维护、可扩展、可审计”的时点。[1][2]

Key Findings

- **主链可用，但不是零差异**：Phase 1 与 Phase 3 的主链闭环已经基本收口；v1 级别仍有正式对象与工程层残差，最典型的是 `deliberation_resolution` 还没有落成正式对象。[1][3][4][22]
- **直接源项目 parity 已发生语义漂移**：Kotlin parity gate 当场失败，7 个 toolprompt 文案和 Kotlin 不再一致；这不是纯文档问题，而是文件系统工具的权限/作用域语义已经变了。[6][15][18]
- **源项目基线本身不稳定**：Kotlin 当前 `.\gradlew.bat test` 在 `SseEventParserTest.kt` 编译失败，说明“直接源项目最新 main 分支”今天不是绿基线；相比之下 Python 源仓仍是绿的、而且测试面更广。[19][20]
- **Phase 4 是当前全局最主要的缺口**：写侧已经有 `object-type-registry` 、 `indexes` 、 `timeline` 、 `operation` ；读侧却没有消费这些派生层，导致 Web/overview/inbox 仍按 `list_dir + read_json` 全量扫描。[2][7][8][11][24]
- **性能问题主要来自 O(N) 读写放大**： `get_case_overview/2` 、全局 inbox、scheduler tick、SSE tail 都是“线性扫文件系统 + 重复打开文件”的风格；小规模没问题，但它们不是可平滑扩容的结构。[7][8][9][12]
- **测试通过不等于工程风险已被覆盖**：EUnit 230 绿了，但没有覆盖动态 atom、防止 scheduler 阻塞、index 读侧消费、多 mail 更新等高风险点。[17][23]

Detailed Analysis

1. 审计基线与方法

这次审计没有只看文档，也没有只看代码，而是按下面四层交叉比对：

1. `v1` 总 PRD 与分 phase 设计稿；[1][2]
2. 既有 Phase 对齐审计结论；[3][4]
3. Erlang 当前 working tree 的真实代码与测试；[7][8][9][10][11][12][22][23][24][25]
4. Kotlin sibling + Python origin 的源码、prompt、权限模型与测试门禁结果。[5][6][15][16][18][19][20]

因此本文的结论优先级是：**当前代码事实 > 旧审计文档 > 源项目计划文档**。

2. 与源项目的对齐程度

2.1 与 Kotlin sibling 的对齐

Kotlin 仍然是 Erlang 的直接 parity 对象，这一点并不是推测，而是仓内规则明文写出来的：

- `AGENTS.md` 明确把本仓定义为 Kotlin sibling，并要求 Kotlin parity work 先记 backlog、再跑 parity helper。[5]
- `scripts/kotlin-parity-check.ps1` 直接把 Kotlin repo 当作基线，检查 toolprompt、safe tools、default tools、schema top-level contract。[6]

对齐的部分：

- 默认工具集名称集合仍然一致：`AskUserQuestion / Read / List / Write / Edit / Glob / Grep / Bash / WebFetch / WebSearch / NotebookEdit / lsp / Skill / SlashCommand / TodoWrit`。[13][15]
- OpenAI tool schema 的主集合仍然同源，Responses 路径和 legacy Chat Completions 路径在 Erlang 里都保留了，说明共享 SDK core 没有分叉到不可对照的程度。[5][13][15]

不对齐的部分：

- parity helper 2026-03-09 直接失败在 `edit.txt`，随后人工继续比对可见共有 **7 个 toolprompt diff**：`edit.txt`、`glob.txt`、`grep.txt`、`list.txt`、`read.txt`、`task.txt`、`write.txt`。[6][18]
- 这些 diff 不是措辞细修，而是 **文件系统作用域语义的变化**：Kotlin 仍以 `project root` 为工具根；Erlang 已把 `Write/Edit` 收窄到 `workspace root`，而 `Read/List/Glob/Grep` 变成 `project + workspace` 双域模型。[13][15]
- `safe tools` 默认放行集合已经扩大。Kotlin 的 safe 集合是 `Read / Glob / Grep / Skill / SlashCommand / AskUserQuestion`；Erlang 额外把 `List / WebFetch / WebSearch` 放进 safe 集合。[13][15]

审计判断：

- 如果目标是“逐字逐句 parity Kotlin CLI/SDK core”，那么当前 **没有对齐**。[6][15][18]
- 如果目标是“在 workflow/session workspace 模型下做有意识的产品化扩展”，那么这些差异 **可以解释**，但必须被正式记录成“有意偏离”，不能继续装作 parity 还在。[2][5][13]

2.2 Kotlin 基线自身的可信度

Kotlin 源仓当前并不是一个稳定绿基线：

- 2026-03-09 本地执行 `.\gradlew.bat test`，在 `SseEventParserTest.kt` 处直接编译失败，报错是 `No value passed for parameter 'data'`。[19]
- 对应实现 `SseParser.kt` 中 `SseEvent` 已演进为 `event + data` 双字段，而测试仍按旧单字段写法调用。[15][19]

这意味着：

- **不能直接把 Kotlin 当前 main 当成“无条件真相源”**；
- 之后做 parity 修复时，应该以“最后一份绿色、可解释的 contract”作为基线，而不是盲追最新代码。

2.3 与 Python origin 的对齐

Python `openagentic-sdk` 今天本地门禁仍然是绿的：`Ran 369 tests ... OK`。[20]

它对 Erlang 的意义主要在两点：

1. **测试纪律基线**：Python 源仓在 runtime / tools / CLI / MCP / OAuth / remote tool / session 等维度的测试面明显更大。[20]
2. **产品范围提醒**：Erlang 当前实现集中在 local-first runtime + workflow + web governance；Python 源仓则已经长到了 MCP、OAuth、CLI PTY、Windows CLI e2e 等更宽的面。[5][20]

因此如果把“与源头项目完整对齐”理解成“与 Python 源仓所有功能完全一致”，那么当前 Erlang **显然不对齐**；但这不构成 v1 阶段的直接阻塞，因为当前 v1 PRD 的主问题域本来就不是 MCP/CLI/TUI，而是 case governance。[1][5][20]

3. 对 v1 需求本身的审计

3.1 Phase 1：案卷 / 候选 / 正式任务基础

Phase 1 的主线在既有审计中已经被判定为“本 phase 范围内对齐完成”，当前代码也支持：

- `case` 顶层对象；
- origin `deliberation_round` 建档；
- `monitoring_candidate` 提取与审批；
- `monitoring_task` / `task_version` / `credential_binding`；
- Web 治理页、任务详情页、统一 inbox 入口。[3][5][22]

但站在“整个 v1”而不是“Phase 1 最小 DoD”的角度，仍有两个残差：

- `monitoring_candidate` 状态机依然是简化版，仍然只看到 `inbox_pending -> approved/discarded`，没有把 `extracted / under_review` 单列成正式状态节点。[3]
- `deliberation_resolution` 这个正式对象 **仍未实现**。当前只有 `deliberation_round.links.resolution_id` 这个占位引用；没有 resolution 对象文件、没有 resolution 三件套、也没有 resolution 的 Web/落盘/审计链。[1][22]

第二点是我认为整个 v1 审计里最容易被前几轮 phase 收口掩盖、但实际上仍未关闭的一条结构性缺口。

3.2 Phase 2：监测执行 / fact report / retry / urgent brief

相比 2026-03-08 第二轮审计时“调度未执行化、异常上呈未闭环”的状态，当前代码已经明显向前走了一大步：

- 有正式 `monitoring_run`、`run_attempt`、`fact_report`；[10][24]
- 有 retry path；[10]
- 有 contract 校验与 failure classify；[10]
- 有 `urgent_brief` 对象与内邮投递；[4][24]
- 有 scheduler `run_once` 与周期 tick 逻辑，说明 `schedule_policy` 已经从纯冻结值推进到可执行机制。[9]

所以从“主链是否存在”的角度看，Phase 2 的核心闭环已经不再是硬缺口。

但仍有一个中等强度的制度残差：

- `monitoring_run.links.pack_ids`、`fact_report.links.pack_ids` 依然是空数组占位；当前 create pack 时只回写 `task.links.active_pack_ids`，没有把 pack 关联反写到历史 run/report 对象上。[2][24]

这不会阻断主链，但会让后续“从报告反查属于哪个观察包”的链路偏弱。

3.3 Phase 3：观察包 / 检察 / 复议卷宗 / 启动复议

这是当前 working tree 最完整、也最接近 PRD 的部分：

- `observation_pack` / `inspection_review` / `reconsideration_package` / preview / deferred / consume-by-round 全都有；[4][24]
- `reconsideration_package` 现在有 pack-local version 语义与 display code；[4][24]
- `inspect_observation_pack` 已纳入 revision gate；[4][24]
- `urgent_brief` 已写入 `timeline.jsonl`；[4][24]
- `operation` 已不再只有 happy-path；[4][24]

这部分我同意 round-4 文档给出的判断：**Phase 3 主链已闭合**。[4]

3.4 Phase 4：落盘审计 / 派生索引 / Web 聚合硬化

这是当前全局最关键的未闭合区。

设计稿对 Phase 4 的要求非常明确：

- 允许用派生索引加速 Web 查询；
- 索引不是唯一真相源，但 **Web 查询不应继续现场扫目录拼状态**；
- timeline / operation / registry 都要成为稳定工程层，而不是只在写侧落一下文件。[2]

当前实现是“只完成了一半”：

- **已经有写侧**：
  - `meta/indexes/*.json`；
  - `meta/timeline.jsonl`；
  - `meta/ops/*.json`；
  - `meta/object-type-registry.json`。[7][11][24]
- **但读侧没有消费这些派生层**：
  - `get_case_overview_map/2` 每次直接把 `rounds / candidates / tasks / mail / templates / packs / reviews / packages` 全读一遍；[7]
  - `list_inbox/2` 每次全扫 `cases/*/meta/mail/*.json`；[8]
  - `object-type-registry.json` 当前只写不读；[11]
  - `timeline.jsonl` 当前只写不读；[24]

所以对 Phase 4 的最终判定只能是：**基础设施已铺出，工程闭环未完成**。

4. 当前实现本身的审计

4.1 高危正确性 / 安全问题

A. 动态 atom 创建

`openagentic_case_store_repo_persist:normalize_key/1` 与 `openagentic_case_scheduler_store:normalize_key/1` 都在对 JSON key 做 `binary_to_atom(K, utf8)`。[11]

在 Erlang 里这不是普通“小问题”，而是典型高危点：

- atom 不会被 GC；
- 这些 JSON 文件来自本地文件系统，是可被异常写入、损坏写入、甚至被工具链间接制造异常 key 的；
- 一旦 key 空间失控，就是 VM 级不可恢复风险。

审计判定：P0。

B. `update_mail_state/2` 会批量改整个 case 的 mail

`openagentic_case_store_api_inbox:update_mail_state/2` 先找到了目标 `mail_id`，但后续 `lists:foreach` 遍历的是 **整个 `meta/mail` 目录的全部 JSON**，且没有按 `mail_id` 或 `related_object_refs` 做过滤，直接对每个 `MailObj0` 调 `update_mail_status/4`。[8]

这意味着：

- 把一封 mail 标成 `read`，理论上会把同 case 下所有 mail 都标成 `read`；
- 现有测试之所以没炸，只是因为测试只覆盖了单 mail 场景。[23]

审计判定：P0。

4.2 性能 / 可扩展性问题

A. `get_case_overview` 是典型全量聚合读

`get_case_overview_map/2` 每次都在读完整 case 树：`rounds / candidates / tasks / mail / templates / packs / reviews / packages`，没有任何索引消费，也没有按需裁剪。[7]

在几十个对象时问题不大；在几百上千个对象时，overview 页面天然会变成慢查询。

B. 全局 inbox 是 O(所有 case × 所有 mail)

`list_inbox/2` 通过 `safe_list_dir(cases_root)` 全扫每个 case，再把每个 case 的 `meta/mail/*.json` 全读进来装饰、再过滤状态。[8]

设计稿已经给了 `mail-unread.json` 这种派生索引思路；当前实现却还在现场拼，这使得 **你已经支付了重建索引的写成本，但没有拿到任何读收益**。[2][7][8]

C. 调度器把“due scan”和“真实执行”耦死了

当前 scheduler tick 做的是：

1. `filelib:wildcard(cases/*)`；
2. 每个 case 再 `wildcard(tasks/*/task.json)`；
3. 命中 due 后直接同步调用 `openagentic_case_store:run_task/2`；
4. `run_task/2` 再同步走到 `openagentic_runtime:query/2`。 [9][10]

也就是说：

- 调度扫描是 O(全量 case/task)；
- 真正的 LLM 调用也跑在这条串行链上；
- 一个慢任务就能拖慢下一轮 due scan。

这不是“后续再优化”的量级，而是架构上的串行瓶颈。

D. SSE tail 是 250ms 轮询 + 反复开文件

`openagentic_web_api_sse.erl` 每 250ms `read_file_info -> open -> position -> read -> close` 一轮，对每个连接独立做。[12]

本地 demo 可接受；多会话、多浏览器标签页时，CPU/IO 噪音会明显上来。

4.3 落盘与工程一致性问题

A. operation / timeline 只覆盖了局部流程

当前 `new_operation/4` 与 `append_best_effort/2` 的使用几乎集中在 `openagentic_case_store_api_reconsideration.erl`，再加上 run success 里的 urgent brief timeline。[24]

这意味着：

- candidate approve / discard；
- task revise / activate；
- credential binding rotate / invalidate；
- 普通 monitoring run 成功/失败的大部分生命周期；

都还没有纳入统一 operation/timeline 语义。

所以 operation/timeline 现在更像“Phase 3 局部增强”，还不是“Phase 4 的全局工程层”。

B. object registry 只写不读

`object-type-registry.json` 已经会被维护 `objects` 与 `type_counts`，但当前代码中没有任何消费方。[11]

这使它暂时只承担了写放大，没有承担任何查询收益。

4.4 contract drift 与 tooling debt

A. parity helper 已过时

`scripts/kotlin-parity-check.ps1` 仍在读取：

- `openagentic_permissions.erl` 里的 `safe_tools/0`；
- `openagentic_runtime.erl` 里的 `default_tools/0`；
- `openagentic_tool_schemas.erl` 里的 monolithic `tool_params`；

但当前实现这些职责已经分散到：

- `openagentic_permissions/openagentic_permissions_policy.erl`；
- `openagentic_runtime/openagentic_runtime_options.erl`；

- `openagentic_tool_schemas/*` 子模块。 [6][13][14]

所以即便把 prompt diff 修完，helper 也仍然不再是可靠 gate。

B. `Glob/Grep` 的 prompt / schema / implementation 三方不完全一致

当前：

- prompt 文案写的是 `root (or path)`；[13][15]
- implementation 也接受 `path` alias；[14][15]
- 但 OpenAI schema 里只暴露了 `root`，没有暴露 `path`。[14][15]

这属于典型的 contract drift：模型看 description 可以学到 `path`，但 schema 未必允许它发出来。

这不是 source parity 独有问题，Kotlin 侧同样存在这个口径裂缝；但对 Erlang 来说，它仍然是当前实现本身应修的缺陷。[14][15]

5. 测试套件审计

5.1 当前门禁状态

- Erlang：`rebar3 eunit` 重新验证为 `230 tests, 0 failures`。[17]
- Kotlin：`.\gradlew.bat test` 当前编译失败，无法作为“今天的绿色直接基线”。[19]
- Python：`python -m unittest -q` 通过，`369 tests, OK`。[20]

5.2 Erlang 测试的优点

- workflow / web / case governance 测试面已经很厚，尤其是近几轮围绕 reconsideration 的对象链和 Web API 已有针对性回归。[4][17]
- 对 Responses/Chat、Anthropic SSE/parser、session store、FS tools、skills、web runtime 等核心面都有覆盖。[5][17]

5.3 Erlang 测试的缺口

以下高风险点目前没有看到明确回归：

- 动态 atom 风险；
- scheduler 不应同步阻塞整轮 tick；
- overview/inbox 读侧应消费索引而不是继续全扫；
- `update_mail_state/2` 在多 mail 情况下只更新目标 mail；
- operation/timeline 是否覆盖了所有关键写动作；
- 性能基准 / 规模化 fixture / 读写放大回归。

因此 230 绿更多代表“功能闭环可用”，还不代表“工程硬化已完成”。

Areas of Consensus

- **共识 1：主业务链路已经能用。** 从立案、抽候选、批任务、跑监测、产事实、做检察、组卷宗到启动复议，这条链今天是通的。[4][17]
- **共识 2：当前最大的剩余工作不是功能缺席，而是工程硬化。** 尤其是 Phase 4 的索引消费、timeline/operation 全覆盖、读写放大治理。[2][7][8][24]
- **共识 3：Kotlin 不再适合作为“盲目逐行追平”的目标。** 它依然是直接 sibling，但今天它自己不是绿基线，而且 Erlang 已经为了 workflow/workspace 模型作出产品化偏离。[15][18][19]

Areas of Debate

- **争议 1：workspace-only write 是“偏离”还是“升级”？** 从 parity 角度看它是偏离；从 workflow 安全边界看它是合理升级。[13][15]
- **争议 2：`deliberation_resolution` 还要不要继续纳入 v1？** 如果继续算 v1，就应实现正式对象；如果不算，就应走明确 ECN，而不是继续靠 `resolution_id` 占位。[1][22]
- **争议 3：是否继续维护“与 Kotlin 精确 parity”的承诺？** 如果保留，就必须修 helper、修 prompts、补偏离清单；如果不保留，就应把 Erlang 的 workspace / governance 语义正式写成自己的 authoritative contract。[5][6][13]

Repair Queue

P0（必须先修）

1. **禁用动态 atom 创建**
   - 方案：JSON decode 后保留 binary key，或仅对白名单 key 用 `binary_to_existing_atom/2`；补异常 key / fuzz regression。[11]
2. **修正 `update_mail_state/2` 只更新目标 mail**
   - 方案：只写命中的 `MailPath`，如确有级联需求则显式按关联对象过滤；补多 mail regression test。[8][23]
3. **把 scheduler 扫描与执行解耦**
   - 方案：tick 只产出 due jobs；执行走 worker/queue；单 task 继续依赖 run-in-progress 防重；补慢 provider 回归。[9][10]

P1（本轮应收口）

4. **让 overview/inbox 先消费 indexes，再决定是否全扫兜底**
   - 方案：先读 `mail-unread.json`、`tasks-by-status.json`、`packs-by-status.json` 等；索引缺失时再 fallback 全扫。[2][7][8]
5. **更新 parity helper 到当前模块布局**
   - 方案：脚本改读 `openagentic_permissions_policy.erl`、`openagentic_runtime_options.erl`、`openagentic_tool_schemas/*`；再决定哪些差异属于 intentional divergence。[6][13][14]
6. **决定 `deliberation_resolution` 去留**
   - 方案 A：实现正式对象 + 挂到 round / package；方案 B：出 ECN，明确 v1 不交付该对象。[1][22]
7. **补 `Glob/Grep` contract consistency test**
   - 方案：同一条测试同时断言 prompt、schema、implementation 的 alias 一致，避免再漂。[14][15]

P2（随后跟进）

8. **扩 operation/timeline 覆盖范围**
   - candidate approve/discard、task revise/activate、binding rotate/invalidate、run success/failure 都纳入统一事件层。[2][24]
9. **优化 SSE 推送模型**
   - 目标：降低每连接 250ms polling 的 IO 噪音；可以考虑 session 级广播器或 file tail worker。[12]
10. **补规模化 benchmark / soak test**

- 至少要验证 100 case / 1000 mail / 500 tasks 量级下 overview、inbox、scheduler 的退化曲线。

Final Judgment

第三轮综合审计的最终结论是：当前 working tree 已经具备“v1 主链可交互、可回归、可继续修”的基础，但还不具备“v1 工程层零差异收口”的条件。

更具体地说：

- **功能主链**：可用，Phase 1~3 基本成形。[3][4][17]
- **直接源项目 parity**：部分对齐，但已出现有意/无意混合漂移，且 parity gate 过时。[6][13][18]
- **源头仓测试纪律**：Python 仍领先，Kotlin 当前不稳定。[19][20]
- **工程硬化**：Phase 4 只完成了一半，当前真正的修复重心应转向安全、正确性和性能，而不是继续横向扩功能。[2][7][8][9][10][11][12]

因此，下一步最合理的动作不是“继续讨论 v1 要不要再加什么”，而是：

1. 先按上面的 `P0 -> P1` 队列收口；
2. 收口后再做一轮只针对 Phase 4 工程层的复审；
3. 到那时再判断能否把整个 v1 对外描述为“已完成”。

Sources

[1] `docs/plans/2026-03-06-case-monitoring-governance-and-reconsideration-prd-v1.md` — v1 总 PRD，一级产品基线。

[2] `docs/plans/2026-03-06-case-monitoring-governance-and-reconsideration-phase-4-storage-audit-and-web-aggregation.md` — Phase 4 对索引、timeline、operation、Web 聚合的正式设计基线。

[3] `docs/plans/2026-03-07-case-monitoring-governance-phase-1-alignment-audit.md` — Phase 1 已收口结论与遗留制度说明。

[4] `docs/plans/2026-03-09-case-monitoring-governance-phase-3-alignment-audit-round-4.md` — Phase 3 第四轮收口审计，说明 reconsideration 主链已闭

合。
[5] `AGENTS.md` — 当前 Erlang repo 的功能面、Kotlin parity 规则、测试策略。
[6] `scripts/kotlin-parity-check.ps1` — 现有 Kotlin parity helper，本轮实测失败。
[7] `apps/openagentic_sdk/src/openagentic_case_store/openagentic_case_store_case_state.erl` — case overview 读取与 index rebuild 的核心实现。
[8] `apps/openagentic_sdk/src/openagentic_case_store/openagentic_case_store_api_inbox.erl` — global inbox 扫描与 `update_mail_state/2` 写路径。
[9] `apps/openagentic_sdk/src/openagentic_case_scheduler/openagentic_case_scheduler_due_scan.erl`、`apps/openagentic_sdk/src/openagentic_case_scheduler/openagentic_case_scheduler_dispatch.erl` — scheduler 的全量扫描与同步派发实现。
[10] `apps/openagentic_sdk/src/openagentic_case_store/openagentic_case_store_run_execute.erl`、`apps/openagentic_sdk/src/openagentic_case_store/openagentic_case_store_api_task_run.erl` — monitoring run 的同步执行路径。
[11] `apps/openagentic_sdk/src/openagentic_case_store/openagentic_case_store_repo_persist.erl`、`apps/openagentic_sdk/src/openagentic_case_scheduler/openagentic_case_scheduler_store.erl` — JSON key 动态转 atom 的实现。
[12] `apps/openagentic_sdk/src/openagentic_web_api_sse.erl` — 250ms polling + reopen file 的 SSE tail 实现。
[13] `apps/openagentic_sdk/src/openagentic_permissions/openagentic_permissions_policy.erl`、`apps/openagentic_sdk/src/openagentic_runtime/openagentic_runtime_options.erl`、`apps/openagentic_sdk/src/openagentic_fs/openagentic_fs_paths.erl`、`apps/openagentic_sdk/priv/toolprompts/*.txt` — 当前 Erlang 的 safe tools、默认工具集与 workspace 语义。
[14] `apps/openagentic_sdk/src/openagentic_tool_schemas/*`、`apps/openagentic_sdk/src/openagentic_tool_glob/openagentic_tool_glob_api.erl`、`apps/openagentic_sdk/src/openagentic_tool_grep/openagentic_tool_grep_api.erl` — tool schema 与执行层 contract。
[15] `E:/development/openagentic-sdk-kotlin/src/main/kotlin/me/lemonhall/openagentic/sdk/tools/ToolPathResolver.kt`、`E:/development/openagentic-sdk-kotlin/src/main/kotlin/me/lemonhall/openagentic/sdk/permissions/PermissionGate.kt`、`E:/development/openagentic-sdk-kotlin/src/main/kotlin/me/lemonhall/openagentic/sdk/tools/OpenAiToolSchemas.kt`、`E:/development/openagentic-sdk-kotlin/src/main/resources/me/lemonhall/openagentic/sdk/toolprompts/*.txt` — 直接 Kotlin parity 基线。
[16] `E:/development/openagentic-sdk-kotlin/docs/plan/v1-index.md`、`E:/development/openagentic-sdk-kotlin/docs/prd/PRD-0003-v3-tools-hooks-provider-parity.md` — Kotlin 源仓的计划/愿景文档，用于理解它的自我定位。
[17] 本地验证（2026-03-09）：`. ./scripts/erlang-env.ps1 -SkipRebar3Verify; rebar3 eunit` → `230 tests, 0 failures`。
[18] 本地验证（2026-03-09）：`./scripts/kotlin-parity-check.ps1` → `Toolprompt content mismatch: edit.txt`，并继续人工比对确认共有 7 个 prompt diff。
[19] 本地验证（2026-03-09）：`E:/development/openagentic-sdk-kotlin` 执行 `.\gradlew.bat test` → `SseEventParserTest.kt` 编译失败。
[20] 本地验证（2026-03-09）：`E:/development/openagentic-sdk` 执行 `python -m unittest -q` → `Ran 369 tests ... OK (skipped=2)`。
[21] 本地状态检查（2026-03-09）：`git status --short` 显示 Erlang working tree 为 dirty，Kotlin/Python working tree 为 clean；本报告因此按“当前工作树事实”而不是“最近提交”立论。
[22] `apps/openagentic_sdk/src/openagentic_case_store/openagentic_case_store_api_case_create.erl` — `deliberation_round.links.resolution_id` 仅为占位引用，未见 `deliberation_resolution` 正式对象落盘。
[23] `apps/openagentic_sdk/test/openagentic_case_store_inbox_test.erl`、`apps/openagentic_sdk/test/openagentic_web_case_governance_library_inbox_test.erl` — inbox 相关测试仅覆盖单 mail 路径，未覆盖多 mail 批量误更新。
[24] `apps/openagentic_sdk/src/openagentic_case_store/openagentic_case_store_api_reconsideration.erl`、`apps/openagentic_sdk/src/openagentic_case_store/openagentic_case_store_ops.erl`、`apps/openagentic_sdk/src/openagentic_case_store/openagentic_case_store_timeline.erl` — reconsideration/operation/timeline 当前覆盖面。
[25] `apps/openagentic_sdk/src/openagentic_case_store/openagentic_case_store_repo_paths.erl` — Phase 4 派生层文件布局（indexes / timeline / ops / registry）的实际落盘路径。

Gaps and Further Research

- 本轮没有额外构造 1000+ 对象规模的基准数据集，因此性能判断以代码路径审计为主，辅以现有门禁时长，而不是完整 benchmark。
- Kotlin 当前不是绿基线，因此“继续对齐 Kotlin”前，最好先冻结一份可验证 contract，而不是直接追它最新 main。
- 如果下一轮要做修复计划，建议先把本报告里的 `P0/P1` 转成单独 implementation plan，并把 `deliberation_resolution` 是否仍属 v1 作为第一个显式决策点。